

Shipper Scoping

How the dashboard's Shippers panel filters the Bronze → Silver pipeline

Repo `bronze_to_silver_conversion` · branch `main` · 19 Aug 2026 · written as implemented and verified against live data, not as a proposal

1 What the feature is

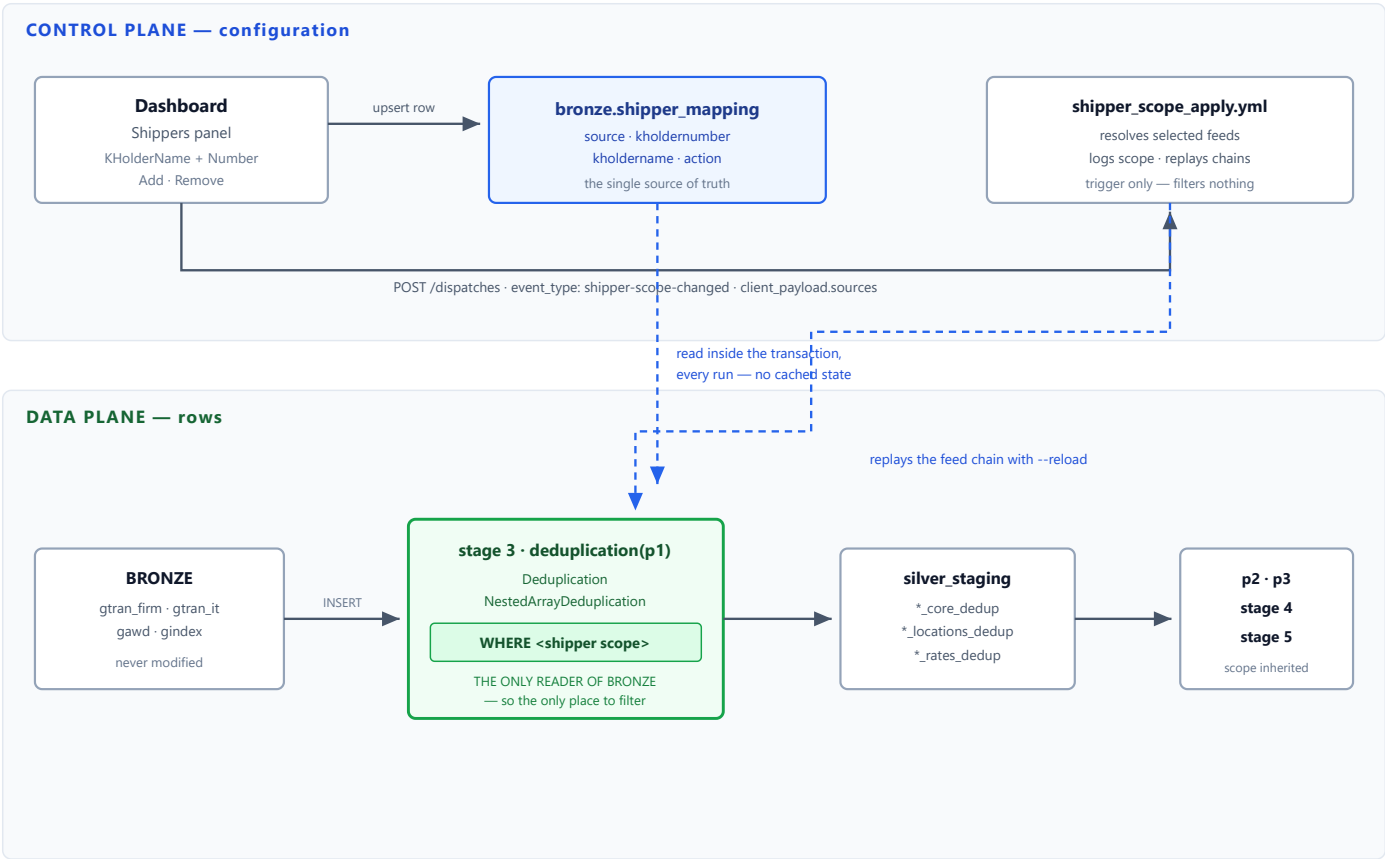
The dashboard's **Shippers** panel attaches shippers to a workflow. Each entry is a **KHolderName + KHolderNumber** pair (the number is a DUNS) with one of two actions:

- **Add** — the raw feed is filtered *down to* that DUNS.
- **Remove** — that DUNS is dropped from the raw feed.

The panel writes config rows. The pipeline reads them and filters. Those are two separate paths that meet in one table, and keeping them separate is the central idea of the design.

2 The two planes

Everything below hangs off this split. The **control plane** is the dashboard writing configuration and asking for a rebuild. The **data plane** is rows moving from Bronze to the master capacity tables. The only coupling between them is the mapping table, read at run time.



Config and rows never touch except at the mapping table. The dashboard writes; stage 3 reads.

THE ONE THING TO CARRY INTO ANY DIAGRAM

The trigger workflow does **not** filter. The filter is a `WHERE` clause inside the stage-3 `INSERT`, because it must be in the same transaction that moves rows out of Bronze. A separate filtering job could only ever delete out-of-scope rows *after* they had already reached staging and been read by the next phase.

3 Data contract — `bronze.shipper_mapping`

The table the dashboard writes and the pipeline reads. It lives in the Bronze schema because it is configuration *for the raw feed*, not a curated output.

COLUMN	TYPE	MEANING
<code>id</code>	<code>bigserial</code> PK	Surrogate key.
<code>workflow_id</code>	<code>integer</code> , null	Which dashboard workflow attached the shipper. Display and audit only — the pipeline never reads it. See §7.
<code>source</code>	<code>text</code> , null	Feed this row scopes: <code>firm</code> , <code>interruptible</code> , <code>awards</code> , <code>ioc</code> , <code>index</code> . <code>NULL</code> means <i>every</i> feed.
<code>kholdernumber</code>	<code>text</code> NOT NULL	The DUNS matched against the feed. Compared with <code>btrim()</code> on both sides.
<code>kholdername</code>	<code>text</code> , null	Display name as typed. Never used for matching.
<code>action</code>	<code>text</code> NOT NULL	'add' or 'remove' (CHECK-constrained, defaults to 'add').
<code>created_at</code> / <code>updated_at</code>	<code>timestampz</code>	Default <code>now()</code> .

Plus one uniqueness rule and one index:

```
CONSTRAINT uq_shipper_mapping_scope
    UNIQUE NULLS NOT DISTINCT (workflow_id, source, kholdernumber)

CREATE INDEX ix_shipper_mapping_lookup
    ON bronze.shipper_mapping (action, source, kholdernumber);
```

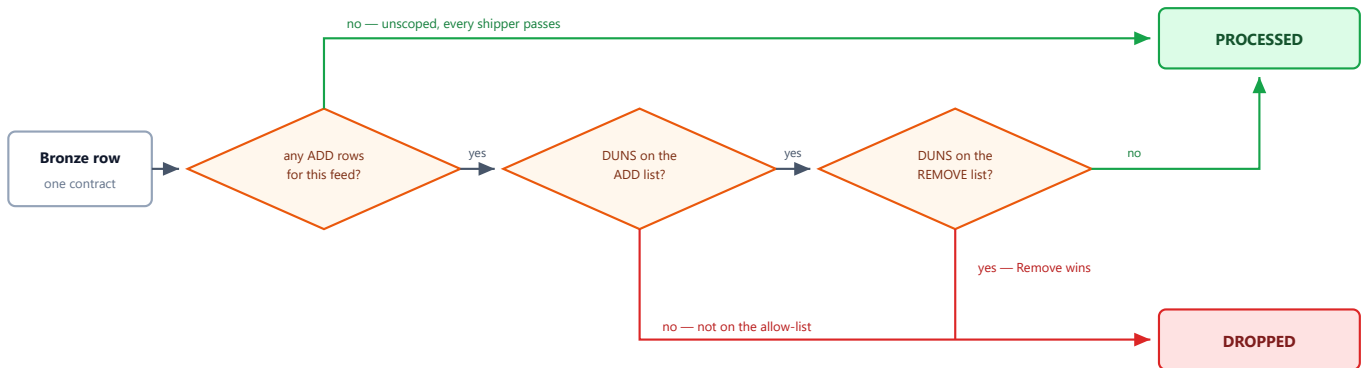
`NULLS NOT DISTINCT` matters: without it, two "all feeds" rows for the same DUNS (`source IS NULL`) would both be allowed to exist. With it, re-adding an existing pair is an **upsert**, not a duplicate — which is what the panel's Add button should map to.

What the frontend does on each action

UI ACTION	WRITE
Add	<code>INSERT ... ON CONFLICT (workflow_id, source, kholdernumber) DO UPDATE SET action='add', kholdername=..., updated_at=now()</code>
Remove	Same upsert with <code>action='remove'</code> . Keeping the row (rather than deleting it) is what makes the exclusion durable and auditable, and matches the panel's copy: <i>"Removing one drops that DUNS from the raw table the next time the feed is processed."</i>
Discard	Not modelled – assumed to discard the unsaved UI row and write nothing. If Discard is meant to <i>delete a previously saved shipper</i> , that is a <code>DELETE</code> and behaves like "never configured", which is different from Remove. Worth confirming.

4 The filter rule

Evaluated per feed, against the mapping table's contents at the moment the run executes.



Green passes, red drops. The top rail is the unconfigured case — the only one live today.

MAPPING-TABLE STATE FOR A FEED	EFFECT
No add rows	Unscoped — every shipper passes. This is the pre-configuration behaviour, so the feature is entirely inert until the dashboard writes a row.
One or more add rows	Allow-list — only those DUNS are processed. Everything else is dropped.
Any remove row	That DUNS never passes, even if also added . Remove always wins.
Only remove rows	Everything passes <i>except</i> the removed DUNS (a deny-list).

Expressed as SQL, this is the whole rule — the `NOT EXISTS` in the first branch is what makes the unconfigured case a genuine no-op:

```
WHERE (
  -- No shipper attached to this feed => unscoped, everything passes.
  NOT EXISTS (
    SELECT 1 FROM bronze.shipper_mapping m
    WHERE m.action = 'add'
      AND (m.source IS NULL OR lower(btrim(m.source)) IN ('firm','firms','gtran_firm'))
  )
  OR EXISTS (
    SELECT 1 FROM bronze.shipper_mapping m
    WHERE m.action = 'add'
      AND (m.source IS NULL OR lower(btrim(m.source)) IN ('firm','firms','gtran_firm'))
      AND btrim(m.kholdernumber) = btrim(s.kholder)
  )
)
AND NOT EXISTS (
  -- An explicit Remove always wins, even over an Add for the same DUNS.
  SELECT 1 FROM bronze.shipper_mapping m
  WHERE m.action = 'remove'
    AND (m.source IS NULL OR lower(btrim(m.source)) IN ('firm','firms','gtran_firm'))
    AND btrim(m.kholdernumber) = btrim(s.kholder)
)
```

The feed-name list (`'firm'`, `'firms'`, `'gtran_firm'`) is generated from the alias table the Parquet export already maintains, so a row written as `firms` or `gtran_firm` still scopes the firm feed instead of silently matching nothing. The frontend does not have to normalise.

5 Where the filter lives, and why only there

`Deduplication` and `NestedArrayDeduplication` (stage 3, phase `deduplication(p1)`) are the **only two classes in the entire pipeline that read a Bronze table**. Phase p2, phase p3, stage 4 and stage 5 all read the staging schema those two write. That single fact is what makes the design small:

- One `WHERE` clause covers **all 7** dedup transformations.
- It covers **every feed** (firm, interruptible, awards, index) and **every grain** (core, locations, rates).
- Every later phase inherits the scope **without knowing scoping exists** — no per-transformation code, nothing to keep in sync.

The nested grains

Locations and rates are not separate Bronze tables; they are JSON arrays nested inside the contract row, exploded with `jsonb_array_elements`. The DUNS lives on the *parent contract*, not inside a location or rate element — so the predicate is evaluated against the parent row, and an out-of-scope contract contributes **no elements at all**. This is why the location and rate counts in §8 fall in step with the core counts.

Per-feed join column

Each feed spells the shipper differently, so the column comes from a lookup (`SHIPPER_KEYS`) rather than being hardcoded to `kholder`:

BRONZE TABLE	FEED	DUNS COLUMN	NAME COLUMN
<code>gtran_firm</code>	firm	<code>kholder</code>	<code>kholdername</code>
<code>gtran_it</code>	interruptible	<code>kholder</code>	<code>kholdername</code>
<code>gawd</code>	awards	<code>bidderduns</code>	<code>biddername</code>
<code>gindex</code>	index	<code>shipperduns</code>	<code>shipper</code>

FAIL-OPEN, DELIBERATELY

A Bronze table missing from that map is **not filtered**, and logs a warning. A feed with no shipper concept should not silently lose every row. Adding a future feed means adding one line to that map and nothing else.

6 Trigger — how a change takes effect

Writing a shipper row changes *configuration only*. The feed keeps whatever scope it was last built with until it is rebuilt. `shipper_scope_apply.yml` is what closes that gap.

```
POST /repos/<owner>/<repo>/dispatches
{
  "event_type": "shipper-scope-changed",
  "client_payload": { "sources": "firm,interruptible" }
}
```

FIELD	CONTRACT
<code>event_type</code>	Always the literal <code>shipper-scope-changed</code> .
<code>sources</code>	Comma-separated feeds the workflow has selected, or <code>"all"</code> . Case and whitespace insensitive. Omitting it replays every feed — always correct, only costs time.

The workflow then, in order:

1. Resolves the feed list (dispatch payload, or the manual-run input, or `all`).
2. Prints the scope about to be applied, per feed, via `python run.py --shippers --source <feed>` . This is the audit trail: the run that rebuilt a feed shows exactly which DUNS were allowed and which were dropped.
3. Runs one gated job per selected feed, each calling that feed's existing `<feed>(stage3_4_5).yaml` chain. Nothing about the chain is duplicated — editing a stage's own file changes what a re-scope runs too.

Those chains already run with `--reload` , which drops and rebuilds the staging and Silver tables. That rebuild re-evaluates the mapping table, so the new scope lands with **no extra logic anywhere**. Any number of feeds can be named; each has its own independently gated job, which is what makes "however many sources are selected" work without a new CLI flag.

ONE DELIBERATE COST

The chain re-ingests Bronze before stage 3. Bronze is untouched by a scope change, so this is an idempotent no-op re-fetch — but it does add time. It was chosen so a re-scope and a normal run are the *same* code path rather than two that can drift apart. Skipping straight to stage 3 is a small change if the extra minutes matter.

7 Scoping boundary — read this before diagramming

SCOPE IS PER FEED, NOT PER WORKFLOW

The panel reads as though shippers attach to *a workflow*. As built, they attach to a **feed**. `workflow_id` is stored but the filter never reads it.

Concretely: if workflow A attaches a shipper to the firm feed, that scope applies to **every** firm run — workflow B's run, the nightly ingest, a manual CLI run, all of it. There is one `silver_staging.firm_core_dedup` table, so there is one answer to "which shippers are in scope for firm."

This was the deliberate trade for a design that needs no changes to the 24 stage workflow files and no `--workflow-id` threaded through every dispatch payload. It is correct as long as **one shipper list per feed** is the real requirement.

Making it genuinely per-workflow is not a small change. It needs either separate staging tables per workflow, or a `workflow_id` column carried through stage 3 → 4 → 5 and into the master capacity tables, plus `--workflow-id` threaded through every workflow file and dispatch payload. Worth doing only if two workflows will really scope the same feed differently. **This is the open question the frontend design should settle.**

8 Verified behaviour

Measured end to end against live Neon data — the firm feed, 2,000 Bronze contract rows, running the real `deduplication(p1)` phase with `--reload` between each state.

MAPPING-TABLE STATE	CORE	LOCATIONS	RATES	CHECK
Nothing configured	2000	4467	3813	Identical to pre-change baseline
<code>add</code> × 2 shippers	100	222	186	= 51 + 49, the two shippers' exact contract counts
<code>add</code> × 2 + <code>remove</code> × 1 (same DUNS)	51	108	96	Remove wins over Add
<code>remove</code> × 1 only, <code>source = NULL</code>	1949	4359	3717	= 2000 – 51; deny-list across all feeds

Database was returned to the unscoped baseline afterwards; the mapping table is currently empty.

9 File map

FILE	ROLE
<code>src/core/shipper_scope.py</code>	Owns the rule. Builds the SQL predicate, the mapping-table DDL, the per-feed column map, and the <code>--shippers</code> report. Holds no DB connection, so <code>--show-sql</code> works without a driver.
<code>src/transformations/stage_3/deduplication(p1)/dedup_base.py</code>	Applies the predicate, on both the flat and the nested-array dedup paths. The only edit needed in the transformation layer.
<code>.github/workflows/shipper_scope_apply.yml</code>	The trigger. Resolves feeds, logs the scope, replays the selected feeds' chains. Filters nothing.
<code><feed>(stage3_4_5).yaml</code> × 3	Gained a <code>workflow_call</code> : trigger so the above can reuse them instead of duplicating the job graph.
<code>run.py</code>	Gained <code>--shippers</code> (optionally <code>--source <feed></code>) to print the live scope.

10 Open questions for the frontend

1. **Per-feed or per-workflow scope?** (§7) The one decision that changes the architecture.
2. **What does Discard do?** Abandon an unsaved row, or delete a saved shipper? The second is a third distinct state alongside Add and Remove.
3. **Does the panel know which feeds a workflow selected?** It must, to populate `source` on write and `sources` in the dispatch payload. Writing `source = NULL` means "all feeds", which is a much broader statement than the panel's wording implies.
4. **Is a KHolderNumber validated against the feed before saving?** A typo'd DUNS silently produces an allow-list that matches nothing — which drops the entire feed. Worth a lookup against `bronze.gtran_firm` at entry time.